Lab 2 Analysis — WalkMates

Course: Principles & Practices in Software Testing (DV033G)
Pair: Lena Edlind, Jakob Cederblad

Part A — Structural Testing & Coverage (M3)

Activity 3.1 — Measure Baseline Coverage

Target: `com.walkmates.service.PricingCalculator` (FR-4.3)
Initial test suite: `PricingCalculatorStructuralTest` (only the `shortWalkPrice` test case)

JaCoCo Baseline Measurements:

- **Instruction Coverage:** 83% (56 of 67 instructions covered, 11 missed)
- **Branch Coverage:** 50% (5 of 10 branches covered, 5 missed)
- **Line Coverage:** 80% (12 of 15 lines covered, 3 missed: line 35, 41, 50)
- **Complexity (Cxty):** 37% (3 of 8 covered, 5 missed)

Identified Uncovered Branches and Lines:

1. **Null-guard branch (lines 34–35):**
   `if (booking == null || listing == null || seeker == null)`
   - Because the condition uses `||` (OR), each check creates a separate branch:
     - `booking == null`
     - `listing == null`
     - `seeker == null`
   - Since all three parameters were non-null in the initial test, none of these 3 true-branches were executed (3 missed branches).
2. **Free-listing path (lines 40–41):**
   `if (baseRate == 0.0)`
   - Because `DOG_WALK` was used, the base rate was 80.0 SEK/h (> 0.0). The branch where `baseRate == 0.0` (e.g., for `SHELTER_VOLUNTEER`) was never executed and the return statement `0.00` (line 41) was not reached. 1 branch missed.
3. **Overnight-surcharge path (lines 49–50):**
   `if (booking.getDurationMinutes() >= OVERNIGHT_THRESHOLD_MINUTES)`
   - The initial test used a booking duration of 60 minutes, which is below the threshold of 480 minutes (8 hours). Therefore, the branch applying the 20% surcharge (line 50) was never executed. 1 branch missed.

Activity 3.2 — Raise Branch Coverage

Test cases implemented for 100% branch coverage:

1. `shouldRejectNullBooking()` — Asserts that `booking == null` throws an `IllegalArgumentException`.
2. `shouldRejectNullListing()` — Asserts that `listing == null` throws an `IllegalArgumentException`.
3. `shouldRejectNullSeeker()` — Asserts that `seeker == null` throws an `IllegalArgumentException`.
4. `shelterVolunteerIsAlwaysFree()` — Asserts that a `SHELTER_VOLUNTEER` listing (baseRate == 0.0) costs 0.00 SEK regardless of duration.
5. `overnightBookingIncludesSurcharge()` — Asserts that a booking clearly exceeding the threshold (e.g., 600 minutes) includes the 20% overnight surcharge.

JaCoCo Measurements After Activity 3.2:

- **Instruction Coverage: 100%** (67 of 67 instructions covered, 0 missed) — *increased from 83%*
- **Branch Coverage: 100%** (10 of 10 branches covered, 0 missed) — *increased from 50%*
- **Line Coverage: 100%** (15 of 15 lines covered, 0 missed) — *increased from 80%*
- **Complexity (Cxty): 100%** (8 of 8 covered, 0 missed) — *increased from 37%*

Activity 3.3 — Coverage ≠ Correctness (Boundary test at 480 minutes)

Requirement (FR-4.3):
The 20% overnight surcharge should **only** be applied when the booking duration is **strictly greater than 480 minutes** (`> 480`).

- **Test Case:** A booking of exactly 480 minutes (8 hours) for a `DOG_WALK` (80 SEK/h) by a `VERIFIED` Seeker (12% platform fee).
  - Expected: No overnight surcharge should be applied.
  - Base price: $8 \times 80 = 640.00$ SEK.
  - Surcharge: 0.00 SEK.
  - Fee (12%): $640.00 \times 0.12 = 76.80$ SEK.
  - Expected Total Price: **716.80 SEK**.

Observed Test Failure:

```
[ERROR] Failures:
[ERROR]   PricingCalculatorStructuralTest.bookingJustBelowSurchargeBoundaryDoesNotIncludeSurcharge:100
expected: 716.8
 but was: 860.16
```

Analysis — Why did 100% coverage miss this bug?

This illustrates a classic "double error" between development and structural testing:

1. **Developer fault:** The developer deviated from specification FR-4.3 by implementing `>=` instead of `>` (`booking.getDurationMinutes() >= OVERNIGHT_THRESHOLD_MINUTES`), thus applying the 20% surcharge at the exact boundary of 480 minutes.
2. **Tester gap in structural testing:** In Activity 3.2, the branch was tested using a non-boundary value (600 minutes). Because 600 minutes successfully executed the surcharge calculation, JaCoCo marked the branch as 100% covered (green).

The coverage tools only check if a line of code was run, not if the math or logic on that line is actually correct. Since our 600-minute test ran the surcharge code, JaCoCo was happy and gave us 100% branch coverage. But JaCoCo didn't know that the code used `>=` instead of `>` for the 480 limit. This shows that having 100% coverage doesn't mean your code is bug-free — you still need to test the exact boundary values yourself to catch mistakes like this.

Resolution:

Corrected line 49 in `PricingCalculator.java` from:

```
if (booking.getDurationMinutes() >= OVERNIGHT_THRESHOLD_MINUTES)
```

to:

```
if (booking.getDurationMinutes() > OVERNIGHT_THRESHOLD_MINUTES)
```

Part B — Test Optimization (M4)

Activity 4.1 — Mutation Testing (PIT)

We ran PIT mutation testing to see how strong our tests are.

1. Results for PricingCalculator:

- **Mutation Coverage:** 100% (14 out of 14 mutants killed).
- PIT tried to break our code by changing the `>` sign to `>=` for the overnight limit. Because we added the exact 480-minute boundary test in Activity 3.3, our test failed exactly when it should, killing the mutant. This proves our tests are strong enough to catch off-by-one errors.

2. Surviving Mutants and the Booking Limit Bug:

When looking at `BookingService.java`, PIT showed surviving mutants marked as `NO_COVERAGE`:

- Line 71: `if (seekerActive > seeker.getMaxConcurrentBookings())`
- Mutant: `changed conditional boundary`

This points to the bug we found in Lab 1:

- A `NEW` seeker is only allowed 1 active booking.
- If they already have 1 active booking (`seekerActive = 1`), the code checks if `1 > 1`. This is false, so it doesn't stop them and they can make a second booking.
- The code should use `>=` so that if you are already at the limit, you get blocked.

Activity 4.2 — Component Isolation with Mocking

We used Mockito to isolate and test `SeekerService` and `BookingService` without needing a real database or real external payment gateway.

1. SeekerService (`SeekerServiceTest`):

We tested the `topUp` method against the three possible payment outcomes:

- **Success:** The payment gateway accepts the charge → the seeker's wallet balance increases, and the seeker is saved.
- **Decline:** The payment gateway throws `PaymentException` → the wallet is not credited, and no save happens.
- **Timeout:** The payment gateway throws `PaymentTimeoutException` → the wallet is not credited, and no save happens.

2. BookingService (▶ `BookingServiceTest`):

- **Confirmation notification:** We mocked all repositories and verified that `notifications.sendBookingConfirmed()` is called when a booking succeeds.
- **Resolving the booking limit bug from Lab 1 (FR-4.4 Rule 2):** We added an equality boundary test for a `NEW` seeker who already has 1 active booking. With the original code (`seekerActive > max`), the condition `1 > 1` evaluated to false, allowing the second booking. After seeing our test catch this failure, we corrected the code in `BookingService.java` from `>` to `>=` so seekers at their limit are properly rejected.

Results in PIT After Adding Mock Tests:

- **Total Project Mutations Killed:** Increased from 67 (35%) to **106 (55%)**.
- **com.walkmates.service package:**
  - `PricingCalculator` : 100% line coverage (15/15), 100% mutation coverage (14/14 killed).
  - `BookingService` : 93% line coverage (39/42), 58% mutation coverage (11/19 killed). The boundary mutant on line 71 is now killed.
  - `SeekerService` : 77% line coverage (10/13), 40% mutation coverage (2/5 killed).

Activity 4.3 — Regression Selection & Prioritization

Scenario: A developer proposes a change on branch `feature/weekend-surcharge` which adds a weekend surcharge by editing `PricingCalculator.java` .

Instead of running all 49 tests every time, we select and prioritize which tests actually need to run to get fast feedback (fail-fast).

1. Change-Relevant Tests (Must Re-run):

1. `PricingCalculatorStructuralTest` **(Priority 1):**
   - **Why:** This directly tests `PricingCalculator` . These unit tests run in less than 20 milliseconds without any database or Spring context. If the new weekend math breaks existing pricing rules (like free listings or the 480-minute boundary), we will know almost instantly.
2. ▶ `BookingServiceTest` **(Priority 2):**
   - **Why:** `BookingService` is the only service that calls `pricing.priceFor()` . It uses the price to check if the seeker has enough money and to charge their wallet. We need to make sure the service still works with the pricing calculator.

2. Wasteful Tests (Do NOT need to run for this change):

- `MatchControllerWebTest` **(Biggest waste):** This test starts up the entire Spring Boot server and takes 3 to 5 seconds to run. It only tests the web controller for AI match explanations and has nothing to do with pricing calculations.
- `MatchExplanationServiceTest` **:** Tests prompt generation for the AI LLM feature. Completely unrelated to pricing.
- ▶ `SeekerSpecBasedTest` **:** 31 tests checking input validation (email format, Swedish phone numbers, etc.) in the `Seeker` class.
- `SeekerServiceTest` **:** Tests adding money to a wallet and handling payment gateway errors. Does not calculate booking prices.
- `BeginnerFirstTest` **:** Initial sanity test from day one.

3. Execution Order & Justification:

We should run the tests in this order:

1. `PricingCalculatorStructuralTest` (~20 ms)
2. ▶ `BookingServiceTest` (~1 sec)

Justification: By running the fastest, directly affected unit tests first, the developer gets feedback in under 1 second. Leaving out the slow Spring web tests saves both time and CI resources.